

应诺尔产品接口文档

产品 API 接口指导说明书

一、接入必读

1.1 适用范围

本文档的适用对象为接入平台的第三方产品技术人员、日常维护人员。

1.2 合作公司须知

1. 正式访问地址接入时提供
2. 合作公司在接入时，需提供外网ip，以便将贵公司 ip 放入白名单
3. 合作公司在接入时，需要先申请开通公司账号

1.3 接口说明

通信协议

接口请求方式为 POST

采用HTTPS 协议进行请求和响应的处理，接口请求及响应返回均为 JSON 格式的数据

接口未注明处均使用UTF-8 编码

注意事项

以下使用的个人数据纯属虚构，仅表示格式，如有雷同，纯属巧合

响应时间

接口超时时间为 4 秒

签名

调用时需根据 body 中的参数以及商务提供的 appId 和 secret 对参数进行签名处理，具体签名

方法可参考 3.1【加签方式】

环境说明

正式环境:正式账号使用，调用已开通接口无限制，按成功条数收费

测试环境:测试账号使用，调用已开通接口返回挡板数据

账号说明:正式账号仅适用于正式环境，测试账号仅适用于测试环境

1.4 请求结构

参数名称	示例值	类型	是否必填	参数说明
appId	y89098io	String	是	appId【由商务分配】
sign	457fa900b705291ada 66bd90f6fef82d	String	是	签名
encryptionType	2	Int	否	1: 明文（默认），2: MD5
apiKey	产品编号	String	是	固定: blackIntV35
body		Json	是	接口请求体，见各接口详细定义

1.5 响应结构

参数名称	示例值	类型	参数说明
code	0	Number	响应编码 0:成功
msg	成功	String	返回文字描述
seqNo	vu6pjf4g0400	String	交易流水号
data		Object	接口数据体，见各接口详细定义
- busiCode	10	int	业务响应 10:查得数据，计费标识
- busiMsg	查询成功	String	业务响应消息
- result		JSON	响应内容

1.6 全局返回码 code 说明

code 返回码	说明
0	成功
-1	响应异常

busiCode 返回码	说明
10	查询成功【计费】
1000	未查得【计费】
1001	账户余额不足
1002	账户信息不存在
1003	appId 异常
1004	产品编号异常
1005	账号信息异常
1006	透支余额已达上限
1007	数据请求异常
1009	服务尚未开通

二、黑名单因子 V35

2.1 接口地址

https://{{域名}}/enol/api/v1/doCheck

测试域名：testenol.cn

生产域名：api.enolfax.com

2.2 请求参数

body 参数说明

参数名称	示例值	类型	是否必填	参数说明
name	刘小龙	String	是	姓名
mobile	13768090912	String	是	手机号
idCard	360428199007084331	String	是	身份证号
tradeNo	457fa900b705291ada6	String	否	业务单号

请求示例

```
{
  "appId": "y89098io",
  "sign": "0528999dd55c025b8f36fc72dceb1f63",
  "apiKey": "blackIntV35",
  "encryptionType": 2,
  "body": {
    "name": "5599cb555474c15138f334e3eddea8a8",
    "idCard": "c0741ab700589df0cf6bbc7970ce4b2a",
    "mobile": "85c526f4cdabfbd1c00770f990e82c1e",
    "tradeNo": "025b8f36fc72dce"
  }
}
```

2.3 响应内容

响应示例

```
{
  "code": 0,
  "msg": "请求成功",
  "data": {
    "busiCode": 10,
    "busiMsg": "success",
    "result": {
      "hit_grade": "2",
      "whether_hit": "1",
      "hit_type": [
        {
          "m1": "0.000",
          "m3": "0.000",
          "m6": "0.000",
          "section": "P1"
        },
        {
          "m1": "0.000",
          "m3": "0.000",
          "m6": "0.000",
          "section": "P2"
        },
        {
          "m1": "0.000",
          "m3": "0.000",
          "m6": "0.000",
          "section": "P3"
        },
        {
          "m1": "0.000",
          "m3": "0.000",
          "m6": "0.000",
          "section": "P4"
        },
        {
          "m1": "0.000",
          "m3": "1.001",
          "m6": "1.005",
          "section": "P5"
        }
      ]
    }
  }
}
```

```
{
  {
    "m1": "0.000",
    "m3": "0.000",
    "m6": "0.000",
    "section": "P6"
  },
  {
    "m1": "0.000",
    "m3": "0.000",
    "m6": "0.000",
    "section": "P7"
  },
  {
    "m1": "0.000",
    "m3": "0.000",
    "m6": "0.000",
    "section": "P8"
  }
]
},
"seqNo": "11vkupc90js00"
}
```

result 参数说明

参数名称	类型	参数说明
whether_hit	int	是否命中，0-否，1-是
hit_grade	int	命中等级，（5，4，3，2，1，0）
hit_type	array	
- m1	String	类型+命中次数（格式：*.***）个位数（0 未命中、1 命中） 小数点后三位表示该类型命中次数 如 1.005，表示命中 5 次
- m3	String	类型+命中次数（格式：*.***）个位数（0 未命中、1 命中） 小数点后三位表示该类型命中次数 如 1.005，表示命中 5 次
- m6	String	类型+命中次数（格式：*.***）个位数（0 未命中、1 命中） 小数点后三位表示该类型命中次数 如 1.005，表示命中 5 次
- section	String	命中场景（根据每个场景展示 m1、m3、m6） P1：（银行） P2：（保险） P3：（汽车金融） P4：（消费金融） P5：（小贷） P6：（特殊金融） P7：（混合金融） P8：（其他金融）

三、加签

3.1 签名方式

1, 筛选并排序获取所有请求的业务参数【body 中的参数】，不包括字节类型参数，如文件、字节流，不包含值为空的参数，并按照第一个字符的键值 ASCII 码递增 排序（字母升序排序），如果遇到相同字符则按照第二个字符的键值 ASCII 码 递增排序，以此类推。

2, 拼接将排序后的参数与其对应值，组合成“参数参数值”的格式连接起来，最后将 secret 拼接到末尾，此时生成的字符串为待签名字符串;如假设业务参数为 name, idCard,mobile【具体按接口规定的业务参数为准】，并都有相应的值，则待签名字符串为：

“name 张三 mobile13290879000idCard330129199511153412secret”

3, 调用签名函数使用各自语言对应 MD5 进行签名。

4, 把生成的签名赋值给 sign 参数，加入到请求参数中。

5, 签名生成示例

【JAVA】


```

public static String sign(Map<String, String> paramValues, String secret) {
    try {
        StringBuilder sb = new StringBuilder();
        List<String> paramNames = new ArrayList<>(paramValues.size());
        paramNames.addAll(paramValues.keySet());
        Collections.sort(paramNames);
        for (String paramName : paramNames) {
            sb.append(paramName).append(paramValues.get(paramName));
        }
        sb.append(secret);
        return genMd5(sb.toString());
    } catch (Exception e) {
    }
    return null;
}

```

【C#】

```

public static string Sign(Dictionary<string, string> paramValues, string secret)
{
    try
    {
        StringBuilder sb = new StringBuilder();
        List<string> paramNames = new List<string>(paramValues.Count);
        paramNames.AddRange(paramValues.Keys);
        paramNames.Sort();
        foreach (string paramName in paramNames)
        {
            sb.Append(paramName).Append(paramValues[paramName]);
        }
        sb.Append(secret);
        return GenMd5(sb.ToString());
    }
    catch (Exception e)
    {
    }
    return null;
}

```

【PHP】

```

function sign($paramValues, $secret) {
    try {
        $sb = "";
        $paramNames = array_keys($paramValues);
        sort($paramNames);
        foreach ($paramNames as $paramName) {
            $sb .= $paramName . $paramValues[$paramName];
        }
        $sb .= $secret;
        return genMd5($sb);
    } catch (Exception $e) {
    }
    return null;
}

```

【python】

```

def sign(paramValues, secret):
    try:
        sb = ""
        paramNames = sorted(paramValues.keys())
        for paramName in paramNames:
            sb += paramName + paramValues[paramName]
        sb += secret
        return genMd5(sb)
    except Exception as e:
        pass
    return None

```